

Item 08 – Agent Frameworks & Tool Use

Pareto Learning Guide (2026)

A Hiring-Oriented Curriculum by Expert Panel

1. Executive Summary

Agents extend LLMs with external tools (search, databases, APIs, code execution). This unlocks new capabilities but introduces complexity. This module teaches agent architecture, tool integration, and reliability patterns.

💡 **Applied AI Engineer:** *Agents are powerful but fragile. Poor implementations loop infinitely, make expensive API calls, or fail silently. Production agents need guardrails, monitoring, and error recovery.*

- Agent architecture patterns (ReAct, function calling)
- Tool integration (APIs, databases, code execution)
- Reliability: retry logic, error handling, timeouts
- Cost control: budgets, rate limiting
- Portfolio: 2 production agent systems

2. The Critical 20%

Concept	Why Critical	Deferred
ReAct Pattern	Standard agent architecture	Advanced planning algorithms
Tool Calling	Interface between LLM and external systems	Custom tool protocols
Error Recovery	Agents fail; handle gracefully	Formal verification
Cost Budgets	Prevent runaway API calls	Advanced resource management
Observability	Debug multi-step agent executions	Distributed tracing systems

3. Engineering-First Deep Understanding

3.1 Agent Architecture: ReAct Pattern

ReAct = Reasoning + Acting. Agent thinks, then acts, then observes result.

Loop:

- 1. Thought: What should I do next?*
- 2. Action: Call tool (search, calculate, query_db)*
- 3. Observation: Tool returns result*
- 4. Repeat until task complete or max iterations*

Example:

Thought: I need to find the latest price of Bitcoin

Action: call_api("coinbase", "BTC-USD")

Observation: {"price": 42350.21, "timestamp": "2026-02-05"}

Thought: I have the price, task complete

Final Answer: Bitcoin is currently \$42,350.21

3.2 Tool Integration Patterns

Define tools with: name, description, parameters, return schema

```
tools = [  
  {  
    "name": "search_web",  
    "description": "Search the web for current information",  
    "parameters": {"query": "string"},  
    "returns": "list of search results"  
  },  
  {  
    "name": "calculate",  
    "description": "Perform mathematical calculations",  
    "parameters": {"expression": "string"},  
    "returns": "number"  
  }  
]
```

```
# Agent decides which tool to use  
agent_response = llm(prompt, tools=tools)  
if agent_response.tool_call:  
    result = execute_tool(agent_response.tool_call)  
    # Feed result back to agent
```

4. Day-by-Day Skill Reconstruction Plan

16-day intensive on agent systems.

- Day 1: Agent Architecture (ReAct pattern)
- Day 2: Tool Calling Basics
- Day 3: Tool Integration (API, DB)
- Day 4: Error Handling & Retries
- Day 5-7: Project - Web search agent
- Day 8: Cost Budgets & Rate Limiting
- Day 9: Agent Observability
- Day 10: Multi-tool Agents
- Day 11: Safety Guardrails
- Day 12-16: Capstone - Production agent system

5. Common Blind Spots

- No iteration limit (infinite loops)
- No cost budgets (runaway API calls)
- Poor tool descriptions (agent uses wrong tool)
- No error recovery (one tool failure kills agent)

6. Self-Assessment Questions

1. Q1: Design guardrails for an agent with database access.
2. Q2: Your agent loops infinitely. Debugging approach?
3. Q3: How do you prevent runaway costs in agent systems?
4. Q4: When should you use agents vs single LLM calls?

Expert Panel Synthesis

Dimension	Score	Evidence
Hiring Relevance	0.80	Agents increasingly common in production
Practical Applicability	0.85	Tool use patterns immediately useful